利用内存模型优化无锁栈

Posted on 2024-01-01 14:46:57 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2aLIpla1ZE4k23o0p0hSxXabSXC)

简介

前文我们通过引用计数实现了无锁并发的栈结构，但是对于原子变量的读，写以及读改写操作默认采用的是 `memory_order_seq_cst` ，`memory_order_seq_cst` 为全局顺序模型，也就是所有线程看到的执行顺序一致，但是这种模型对性能消耗较大，本文在之前实现的无锁栈的基础上介绍如何通过更为宽松的模型提升性能。先带着大家复习一下内存模型相关知识

release-acquire同步

我们在之前的文章介绍了6中内存顺序，其中我们可以通过 `release` 和 `acquire` 的方式实现同步的效果，现在带着大家复习一下：

线程A执行store操作，采用 `memory_order_release` 顺序模型。线程B执行load操作采用 `memory_order_acquire` 顺序模型。如果线程B的load操作读取到A操作的store操作的数值，我们称线程a的store操作 synchronizes-with(同步) 线程b的load操作

happens-before先行

因为a->store 同步于 b->load， 则 a->store 先行于 b->load。

只要同步就能推出先行，所谓先行就是逻辑执行的顺序，一定是a->store 先于 b->load

先行还包括一种情况，sequenced-before(顺序执行)， 所谓顺序执行就是单线程中执行的顺序为从上到下的顺序, 比如

```
1. int func(){
2.     int a = 1008; //1
3.     int b = 1024; //2
4. }
```

单线程角度1先于2执行(1 sequenced before 2)，也可推导出1先行于2.

先行具有传递性 1 happens-before 2， 2 happens-before 3, 则1 happens-before 3

注意先行是C++ 语意层面的概念， 指令实际的执行顺序可能是先2后1，取决于编译器。

但是我们可以通过内存顺序进行约束达到指令编排让1先于2的目的。如release内存序能保证其写操作之前的指令不会排在其后。acquire内存序能保证其读操作之前写入的指令不会排在其之后，也能保证其之后的指令不会排在读之前。所以release和acquire形成同步后类似于屏障，当然C++ 也有类似于的原语std::atomic_thread_fence(栅栏)。

写个代码复习一下

```
1. void TestReleaseSeq() {
2.     int data = 0;
3.     std::atomic<int> flag = 0;
4.     std::thread t1([&]() {
5.         data = 42;  //1
6.         flag.store(1, std::memory_order_release); //2
7.         });
8.
9.     std::thread  t2([&]() {
10.        //3
11.        while (!flag.load(std::memory_order_acquire));
12.        //4
13.        assert(data == 42);
14.    });
15.
16.    t1.join();
17.    t2.join();
18. }
```

我们从两方面解读代码：

1 指令编排角度， 2处使用了release内存序，保证1 会排在 2 之前。 3采用了acquire内存序，保证4排在3之后，且如果3能读到2的写入值，则保证指令1已经先于3执行完。因为while重试的机制，保证2同步于3，即2先于3执行，有因为1先于2执行，而3先于4执行，所以得出1先于4执行，那么4处断言正确不会崩溃。

2 从C++先行语义的角度，单线程t1内，1先行于2, 单线程t2内3先行于4, 而t1第2处代码采用release内存序，t2第3处代码采用acquire内存序列，2同步于3, 则2 先行于 3. 因为先行的传

递性，1 sequenced-before 2, 2 happens-before 3, 3 sequenced-before 4, 则1 happens-before 4.

释放序列的扩展

这段文字摘录于C++并发编程一书

> *如果存储操作的标记是memory_order_release、memory_order_acq_rel或memory_order_seq_cst，而载入操作则以memory_order_consume、memory_order_acquire或memory_order_seq_cst标记，这些操作前后相扣成链，每次载入的值都源自前面的存储操作，那么该操作链由一个释放序列组成。若最后的载入操作服从内存次序memory_order_acquire或memory_order_seq_cst，则最初的存储操作与它构成同步关系。但如果该载入操作服从的内存次序是memory_order_consume，那么两者构成前序依赖关系。操作链中，每个“读-改-写”操作都可选用任意内存次序，甚至也能选用memory_order_relaxed次序。*

我们对上述阐述总结为下面的理解

release-sequnece的概念如下:

针对一个原子变量 M 的 release 操作 A 完成后, 接下来 M 上可能还会有一连串的其他操作. 如果这一连串操作是由

1 同一线程上的写操作

2 或者任意线程上的 read-modify-write(可以是任意内存顺序) 操作

这两种构成的, 则称这一连串的操作为以 release 操作 A 为首的 release sequence. 这里的写操作和 read-modify-write 操作可以使用任意内存顺序.

而同步的概念是:

一个 acquire 操作在同一个原子变量上读到了一个 release 操作写入的值, 或者读到了以这个 release 操作为首的 release sequence 写入的值, 那么这个 release 操作 “synchronizes-with” 这个 acquire 操作

所以release-sequence不一定构成同步，只有acquire到release的值才算作同步。

我们看下面的例子,该例子选取自C++ 并发编程中，我对其稍作修改以保证可以正常运行。

我们先定义了三个全局变量，分别是queue_data表示入队的数据，count表示入队的数量。

store_finish表示存储完成。

```
1. std::vector<int> queue_data;
2. std::atomic<int> count;
3. std::atomic<bool> store_finish = false;
```

我们实现入队逻辑，这个逻辑以后会有一个线程独立执行

```
1. void populate_queue()
2. {
3.     unsigned const number_of_items = 20;
4.     queue_data.clear();
5.     for (unsigned i = 0; i < number_of_items; ++i)
6.     {
7.         queue_data.push_back(i);
8.     }
9.     // 1 最初的存储操作
10.    count.store(number_of_items, std::memory_order_release);
11.    store_finish.store(true, std::memory_order_release);
12. }
```

上述函数将20个元素从0到19依次入队，然后修改count值为20，使用release内存顺序，并且将完成标记设置为true.

然后我们实现消费函数

```
1. void consume_queue_items()
2. {
3.     while (true)
4.     {
5.         //2等待存储完成
6.         while (!store_finish.load(std::memory_order_acquire));
7.
8.         int item_index;
9.         //3 读-改-写"操作
10.        if ((item_index = count.fetch_sub(1, std::memory_order_acquire)) <= 0)
11.        {
12.            return;
13.        }
14.        //4 从内部容器queue_data 读取数据项是安全行为
15.        std::cout << "queue_data is  " << queue_data[item_index-1] << std::endl;
16.    }
17. }
```

上述函数，我们在2处等待存储完成，在3处读改写修改count的值，采用的是acquire内存顺序，然后我们从队列中根据item_index读取数据。

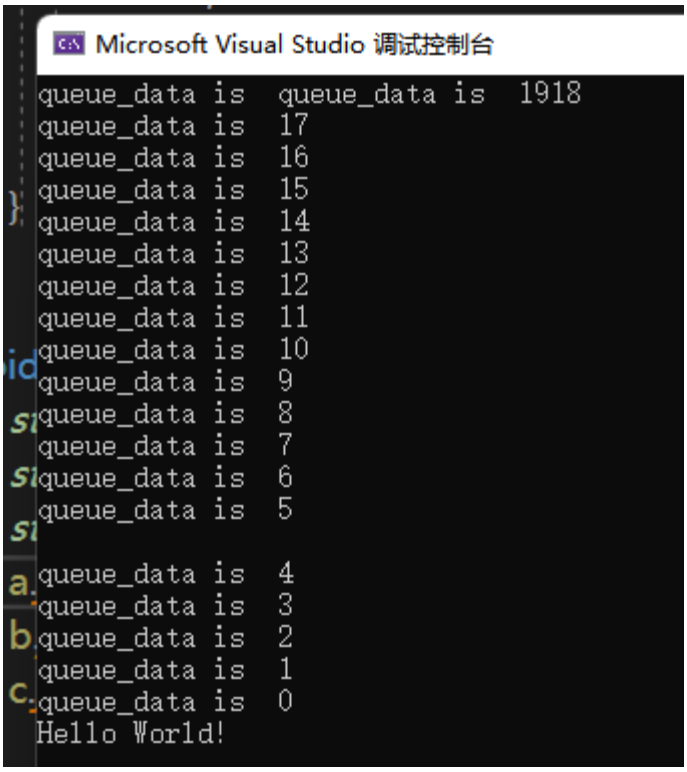
假设一个线程t1用来执行populate_queue，一个线程t2用来执行consume_queue_items。

那么因为release-acquire的关系，我们可以推断出 t1 synchronizes-with t2.

那我们用三个线程并行操作会怎样呢?

```
1. void TestReleaseSeq2() {
2.     std::thread a(populate_queue);
3.     std::thread b(consume_queue_items);
4.     std::thread c(consume_queue_items);
5.     a.join();
6.     b.join();
7.     c.join();
8. }
```

可以看到输出如下

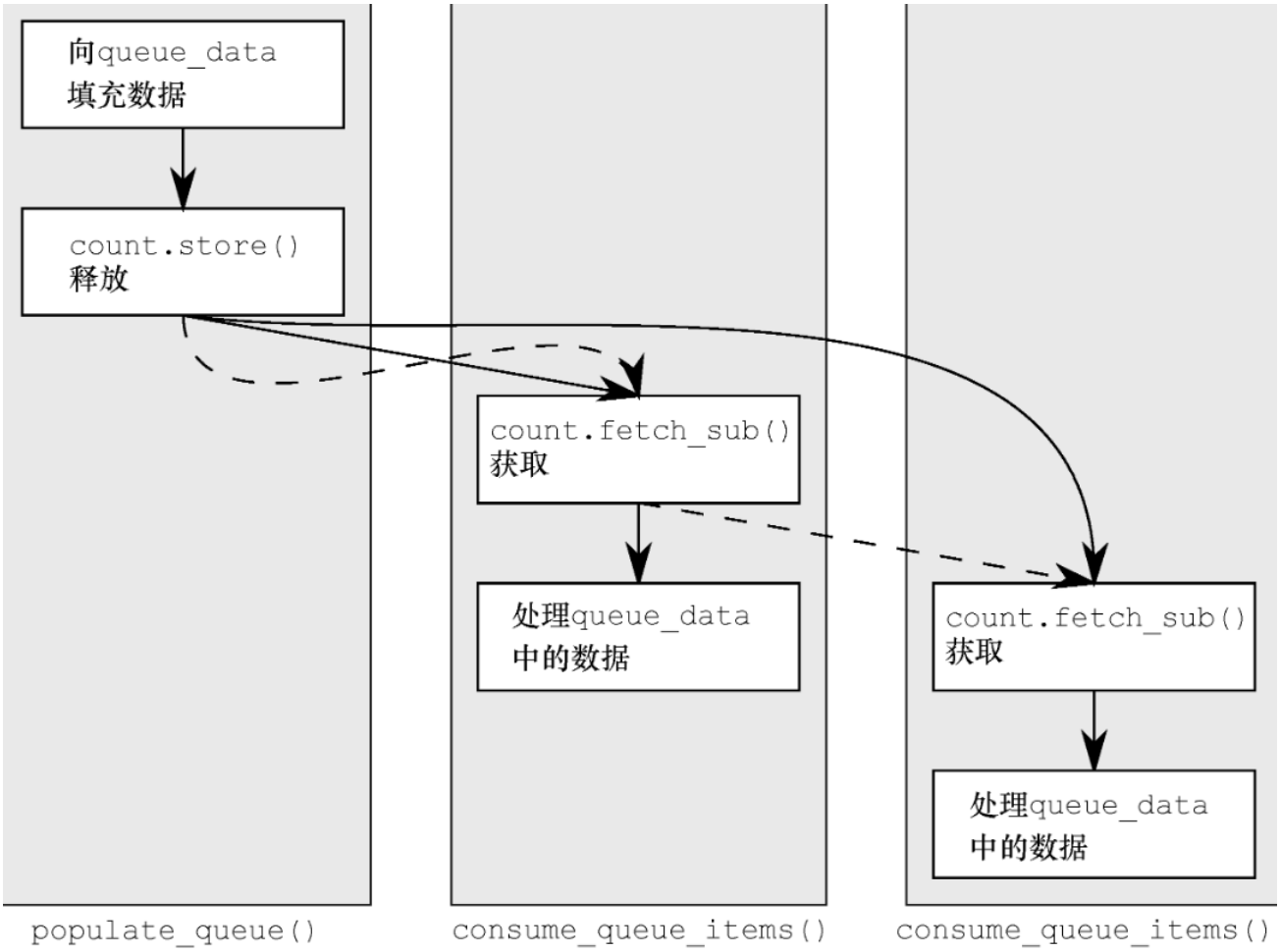


虽然控制台打印混乱，但是我们可以看到消费者线程t2和t3并没有打印重复的数据，说明他们互斥访问count,每个线程取到的count不一样进而访问queue_data中的不同数据。

假设只有一个线程a和线程b,我们知道一个生产者a和一个消费者b构成了同步关系，没有问题，如果增加了消费者线程c，b和c中都有fetch_sub这种读-改-写操作，采用的都是acquire内存顺序，但从线程b和c的角度并不能构成同步，那是不是就意味着b和c可能获取到count的值相同?

答案是否定的，单从线程角度b和c并不能构成同步，但是b和c必然有一个线程先执行一个线程后执行fetch_sub(原子变量的操作任何顺序模型都能保证操作的原子性)。假设b先执行，和a构成release-sequence关系，b读取到a执行的count strore的结果， b处于以a线程的release为首的释放序列中，则b的store操作会和c的读-改-写(fetch操作)构成同步(c 采用的是acquire). C++并发编程一书 (https://book.douban.com/subject/35653912/)中对类似的代码也做了同样的解释。

如下图是书中给出的图示，实线表示先行关系，虚线标识释放序列



那我们可以这么简化上面的分析结论

1 a线程和b线程构成release-sequence的释放序列

2 即使b线程和c线程不构成同步，但是b线程的读改写操作处于release-sequence中，且c线程采用acquire方式读改写，则b的读改写和c线程的读改写构成同步， 以a线程的release为首的sequence序列和c线程的读改写构成同步。

3 这里要强调一点， 如果a relese-sequence b， a和b不一定构成同步，但是b sychronizes with c， 则a synchronizes with c. 简单来说处于relase序列中的任意读改写操作和其他的线程构成同步，那么我们就能得出relese-sequence为首的操作和其他线程同步。

优化无锁栈

我们优化无锁栈先从push操作下手，我们要考虑的是如果有数据入栈，那么pop时要读取最新入栈的数据。所以我们要让push操作同步给pop操作，想到的办法很简单，push对head的修改采用release内存序列，pop对head的读改写采用acquire内存序列。

如果未有元素入栈，多个线程pop并不会产生问题，根据head内部的ptr指向为空判断空栈直接返回空指针。

如果此时已经有一个元素在栈中，多个线程并发pop，执行读改写操作，这些线程本来是无法同步的，但是最先pop的线程会和push线程构成同步关系，且形成release-sequence。那之后的线程pop就会和第一个pop的线程的写操作形成同步。

简单总结上面的含义：

1 因为要保证pop操作时节点的数据是有效的。push和pop要构成同步关系，push 采用release内存序修改head，pop 采用acquire内存序修改head

2 第一个pop的线程的写操作和之后的pop线程读操作要构成同步关系

实现push函数

```
1. void push(T const& data) {
2.     counted_node_ptr  new_node;
3.     new_node.ptr = new count_node(data);
4.     new_node.external_count = 1;
5.     new_node.ptr->next = head.load();
6.     while (!head.compare_exchange_weak(new_node.ptr->next, new_node,
7.     memory_order::memory_order_release, memory_order::memory_order_relaxed));
8. }
```

对于head的修改，我们采用compare_exchange_weak操作。如果修改成功则使用memory_order_release内存顺序，否则就用memory_order_relaxed内存顺序。因为失败会进行重试，所以什么内存序都可以。

接下来实现pop

```
1. std::shared_ptr<T> pop() {
2.     counted_node_ptr old_head = head.load();
3.     for (;;) {
4.         increase_head_count(old_head);
5.         count_node* const ptr = old_head.ptr;
6.         //1  判断为空责直接返回
7.         if (!ptr) {
8.             return std::shared_ptr<T>();
9.         }
10.        //省略数据出栈和头部更新操作....
11.        //....
12.    }
13. }
```

在pop中我们先将head加载出来，然后利用increase_head_count对old_head外部引用技术+1.

我们先讨论increase_head_count的实现,因为我们在increase_head_count的时候很可能其他的线程执行push操作。

因为increae_head_count和push操作都是对head的读改写操作，我们知道无论采用何种内存模型，原子变量的读改写都能保证原子数据的完整性。

因为pop操作比如第1处代码，以至于后面的操作会用到ptr->data数据，所以必须要让push操作和pop操作达到同步关系，才能保证push的data数据对pop操作可见，increase_count用的是acquire模型，而push用的是release模型，保证push先行于pop，这样pop逻辑中的data就是有效的。

```
1. //增加头部节点引用数量
2. void increase_head_count(counted_node_ptr& old_counter) {
3.     counted_node_ptr new_counter;
4.
5.     do {
6.         new_counter = old_counter;
7.         ++new_counter.external_count;
8.     }//7  循环判断保证head和old_counter想等时做更新,多线程情况保证引用计数原子递增。
9.     while (!head.compare_exchange_strong(old_counter,  new_counter,
10.         std::memory_order_acquire, std::memory_order_relaxed));
11.    //8  走到此处说明head的external_count已经被更新了
12.    old_counter.external_count = new_counter.external_count;
13. }
```

接下来我们实现省略的部分，省略的部分要根据head和old_head的值是否想等做出不同的逻辑，上一篇的逻辑是：

1 如果head和old_head相等则说明本线程抢占了head并且需要对外部引用计数-2，得出其他线程增加的引用计数，如果这个引用计数为内部的引用计数(可为负数)的负数则说明其他线程已经不再占有head，已经做了内部引用计数的更新，本线程回收资源即可。

2 如果head和old_head不相等则说明head已经被更改，或者自己获取的old_head是旧的(两个线程并发执行pop, 该线程是引用计数不准确的那个或者该线程读取的head已经被弹出)，所以只需减少内部引用计数即可。

所以之后的逻辑是这样的

```
1. //2 本线程如果抢先完成head的更新
2. if (head.compare_exchange_strong(old_head, ptr->next))
3. {
4.     //返回头部数据
5.     std::shared_ptr<T> res;
6.     //交换数据
7.     res.swap(ptr->data);
8.     //3 减少外部引用计数，先统计到目前为止增加了多少外部引用
9.     int const count_increase = old_head.external_count - 2;
10.    //4 将内部引用计数添加
11.    if (ptr->internal_count.fetch_add(count_increase) == -count_increase)
12.    {
13.        delete  ptr;
14.    }
15.        return res;
16. }
17. else if (ptr->internal_count.fetch_sub(1) == 1) {//5
18.        //如果当前线程操作的head节点已经被别的线程更新，则减少内部引用计数
19.        //当前线程减少内部引用计数，返回之前值为1说明指针仅被当前线程引用
20.        delete ptr;
21. }
```

对于无锁编程，我的心得有两点

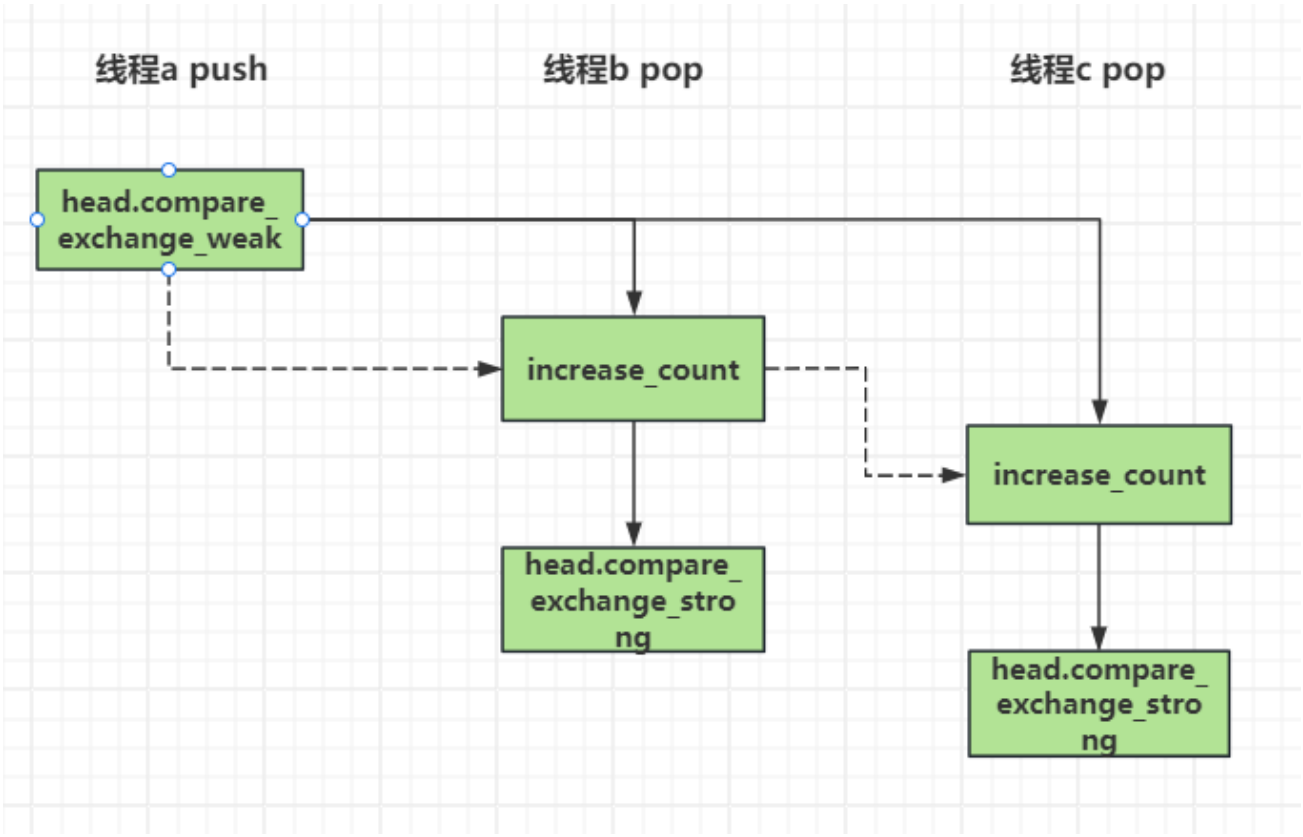
1 对于一个原子变量M，其释放序列中的读改写操作无论采用何种模型都能读取M的最新值。
2 内存顺序模型用来保证数据在多个线程的可见顺序。

例如
因为本线程在2处的比较交换要获取到其他线程修改的head的最新情况，其他线程要么是push操作，要么是pop操作。

1 经过前面的分析，线程a push 操作同步于b线程和c线程的pop中的increase_count操作，那么b线程2处的compare_exchange_strong和c线程2处的compare_exchange_strong都能读取到push操作写入data值。

2 那么b线程2处的compare_exchange_strong和c线程2处的compare_exchange_strong并不构成同步，但他们一定处于释放序列中，因为原子操作读改写保证了原子性。

绘制运行图，实线表示先行，虚线表示释放序列。



所以综上所述2处比较交换采用relaxed即可，大家不放心可以采用acquire方式。

接下来考虑compare_exchange_strong比较成功和失败之后各自内部的逻辑，因为我们要保证ptr的data在被删除之前swap到res里。

1 如果是走入2处的逻辑进入4处代码删除ptr，那么需要保证swap操作先于fetch_add之后的delete操作，所以fetch_add采用release模型。

2 对于5处的fetch_sub操作，内部如果满足删除delete则删除ptr指针，要保证2处逻辑内的swap操作先于delete操作。所以5处的fetch_sub要采用acquire操作。

整理后的pop操作如下

```
std::shared_ptr<T> pop() {
    counted_node_ptr old_head = head.load();
    for (;;) {
        increase_head_count(old_head);
        count_node* const ptr = old_head.ptr;
        //1 判断为空责直接返回
        if (!ptr) {
            return std::shared_ptr<T>();
        }

        //2 本线程如果抢先完成head的更新
        if (head.compare_exchange_strong(old_head, ptr->next,  std::memory_order_relaxed)) {
            //返回头部数据
            std::shared_ptr<T> res;
            //交换数据
            res.swap(ptr->data);
            //3 减少外部引用计数，先统计到目前为止增加了多少外部引用
            int const count_increase = old_head.external_count - 2;
            //4 将内部引用计数添加
            if (ptr->internal_count.fetch_add(count_increase, std::memory_order_release) == -co
    unt_increase) {
                    delete  ptr;
            }
            return res;
        } else if (ptr->internal_count.fetch_sub(1, std::memory_order_acquire) == 1) { //5
            //如果当前线程操作的head节点已经被别的线程更新，则减少内部引用计数
                delete ptr;
        }
    }
}
```

但是并发编程的作者认为5处采用acquire内存序过于严格，可以采用relaxed方式，只要在条件满足后删除ptr时在约束内存顺序即可，为了保证swap操作先执行完，则需在delete ptr 之前用acquire内存序约束一下即可，在delete ptr 上面添加 head.load(std::memory_order_acquire)即可和前面的释放序列构成同步关系。

所以我们最后优化的pop函数为

```
std::shared_ptr<T> pop() {
    counted_node_ptr old_head = head.load();
    for (;;) {
        increase_head_count(old_head);
        count_node* const ptr = old_head.ptr;
        //1 判断为空责直接返回
        if (!ptr) {
            return std::shared_ptr<T>();
        }

        //2 本线程如果抢先完成head的更新
        if (head.compare_exchange_strong(old_head, ptr->next,  std::memory_order_relaxed)) {
            //返回头部数据
            std::shared_ptr<T> res;
            //交换数据
            res.swap(ptr->data);
            //3 减少外部引用计数，先统计到目前为止增加了多少外部引用
            int const count_increase = old_head.external_count - 2;
            //4 将内部引用计数添加
            if (ptr->internal_count.fetch_add(count_increase, std::memory_order_release) == -co
unt_increase) {
                delete  ptr;
            }
            return res;
        } else if (ptr->internal_count.fetch_add(-1, std::memory_order_acquire) == 1) { //5
            //如果当前线程操作的head节点已经被别的线程更新，则减少内部引用计数
            //当前线程减少内部引用计数，返回之前值为1说明指针仅被当前线程引用
            ptr->internal_count.load(std::memory_order_acquire);
                delete ptr;
        }
    }
}
```

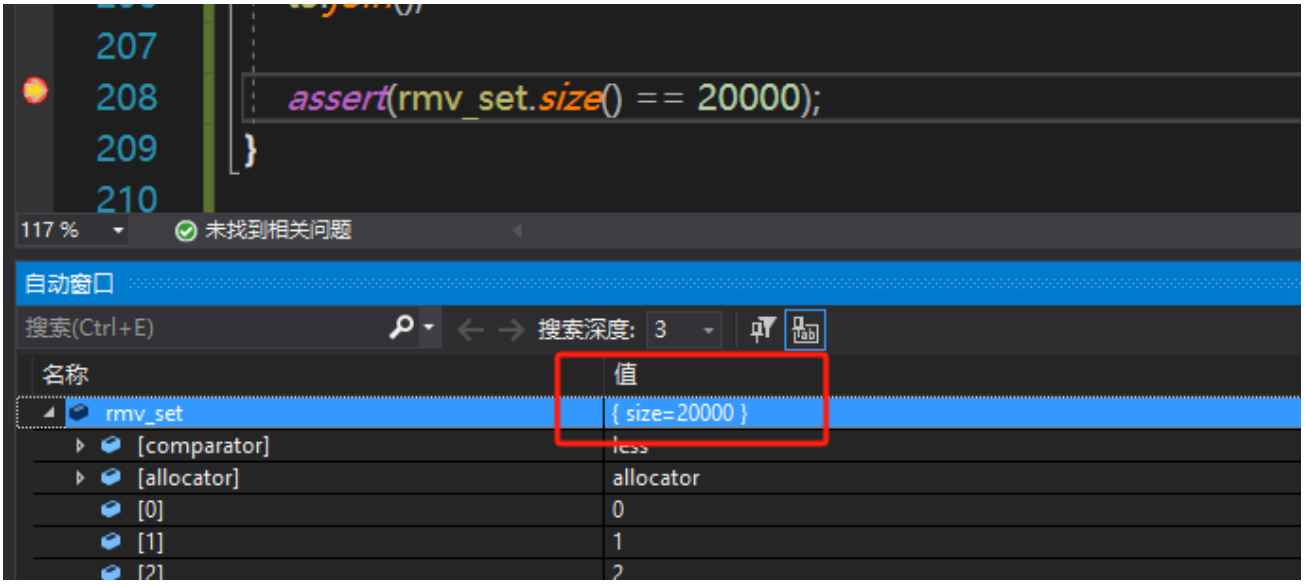
测试

为了保证测试效果，我们还是启动三个线程, t1用来向栈中写入20000个数据，t2和t3分别并发从栈中读取10000个数据放入set中，最后我们看到set的大小为20000个即为正常。

```
1. void TestRefCountStack() {
2.     ref_count_stack<int>  ref_count_stack;
3.     std::set<int>  rmv_set;
4.     std::mutex set_mtx;
5.
6.     std::thread t1([&]() {
7.         for (int i = 0; i < 20000; i++) {
8.             ref_count_stack.push(i);
9.             std::cout << "push data " << i << " success!" << std::endl;
10.            std::this_thread::sleep_for(std::chrono::milliseconds(5));
11.        }
12.        });
13.
14.    std::thread t2([&]() {
15.        for (int i = 0; i < 10000;) {
16.            auto head = ref_count_stack.pop();
17.            if (!head) {
18.                std::this_thread::sleep_for(std::chrono::milliseconds(10));
19.                continue;
20.            }
21.            std::lock_guard<std::mutex> lock(set_mtx);
22.            rmv_set.insert(*head);
23.            std::cout << "pop data " << *head << " success!" << std::endl;
24.            i++;
25.        }
26.        });
27.
28.    std::thread t3([&]() {
29.        for (int i = 0; i < 10000;) {
30.            auto head = ref_count_stack.pop();
31.            if (!head) {
32.                std::this_thread::sleep_for(std::chrono::milliseconds(10));
33.                continue;
34.            }
35.            std::lock_guard<std::mutex> lock(set_mtx);
36.            rmv_set.insert(*head);
37.            std::cout << "pop data " << *head << " success!" << std::endl;
38.            i++;
39.        }
40.        });
41.
42.    t1.join();
```

```
43.     t2.join();
44.     t3.join();
45.
46.     assert(rmv_set.size() == 20000);
47. }
```

我们在assert处打个断点，可以看到集合大小确实为两万个，而且不存在重复元素，不存在缺失的元素。



总结

源码链接：
https://gitee.com/secondtonone1/boostasio-learn/tree/master/concurrent/day17-LockFreeStack (https://gitee.com/secondtonone1/boostasio-learn/tree/master/concurrent/day17-LockFreeStack)

视频链接：
https://space.bilibili.com/271469206/channel/collectiondetail?sid=1623290 (https://space.bilibili.com/271469206/channel/collectiondetail?sid=1623290)

浏览(2329) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) ♡喜欢(552) (JavaScript:void(0);)

热门评论

请输入正文

评论

没有更多评论了,点击收起 ≪ (JavaScript:void(0);)

热门文章

slice介绍和使用

♡喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

♡喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

♡喜欢(587) 浏览(1595)

解密定时器的实现细节

♡喜欢(566) 浏览(1449)

Linux环境搭建和编码

♡喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



恋恋风辰

北京 朝阳



扫一扫上面的二维码图案，加我为朋友。

个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。

京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)

京ICP备2021033081号 (https://beian.miit.gov.cn/)